C语言一维数组

掌握数据集合的基础，开启编程新篇章

1. 一维数组概念

数组：思想精髓类比来新一个集合。在内存中是一段连续的空间。

可以将C语言中的**一维数组**类比现实新入**一个集合**，**一维数组**的**元素**（**一条有序的盒子序列**），每个**相同**的（数组元素）可以存放一种数据（数据），这种有序相同的盒子大小相同（数据类型一致），并通过编号（索引）快速找到对应位置。

如下图所示：

一维数组示意图

0 此处是0下标大小数组

2. 一维数组创建和初始化

一维数组创建数据基本语法如下：

```
type arr_name[元素个数];
```

根据语法我们可以创建出这种初始数组：

示例：

```
int arr[5];
```

表示创建了一个大小为5的整型数组

如何对该数组进行初始化呢？

```
int main()
{
    int arr[10] = {1,2,3,4,5};

    int arr[5] = {1,2};

    //字符数组的长度，程序会根据数组长度个数得出编译长度
    int arr[5] = {1,2};

    return 0;
}
```

3. 一维数组使用

3.1 数组下标

数组下标规则

C语言数组使用下标的，下标最从0开始的，假设数组有n个元素，最后一个元素的下标是n-1，下标就相当于数组元素的编号，如下：

```
int arr[10] = {1,2,3,4,5,6,7,8,9,10};
```

数组	1	2	3	4	5	6	7	8	9	10
下标	0	1	2	3	4	5	6	7	8	9

数组下标示意图

0 此处是0下标大小数组

特别注意数组下标规则：

1. 下标最从0开始的
2. 数组长度为n，最后一个数组的下标是n-1
3. 数组长度下标最大，数组的下标是n-1

3.2 通过数组下标访问元素

访问元素的方式为：

数组名[下标]

3.2.1 访问单个数组

```
#include<stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    printf("%d\n",arr[2]);//输出3
    printf("%d\n",arr[7]);//输出8
    return 0;
}
```

3.2.2 访问数组每个数组元素

```
#include<stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    for (int i = 0; i < 10; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

3.2.3 动态创入元素

```
#include<stdio.h>
int main()
{
    int arr[5];
    int i = 0;
    for (i = 0; i < 5; i++)
    {
        scanf("%d", &arr[i]);
    }
    for (i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

4. 一维数组在内存当中的存储

数组的内存特性

我们知道数组的定义是**相同类型元素的一个集合**，在内存中是一段连续的空间。

那体现在个元素与每个元素之间的内存是连续的。

通过程序打印数组中每个元素的地址：

```
#include<stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int i = 0;
    for (i = 0; i < 10; i++)
    {
        //注意打印地址的时候要加上&，这样可字长打印出来，事实上使用%p也是可以的
        printf("&arr[%d] = %p\n", i, &arr[i]);
    }
    return 0;
}
```

```
输出结果：
&arr[0] = 0140fd0c0
&arr[1] = 0140fd0c4
&arr[2] = 0140fd0c8
&arr[3] = 0140fd0cc
&arr[4] = 0140fd0d0
&arr[5] = 0140fd0d4
&arr[6] = 0140fd0d8
&arr[7] = 0140fd0dc
&arr[8] = 0140fd0e0
&arr[9] = 0140fd0e4
```

观察输出结果我们发现，由于是整型数组，每个元素是4个字节，所以地址间隔也是4。

5. sizeof与数组

sizeof计算数组大小

我们可以通过sizeof数组名/单计算整个数组的字节大小。

```
#include<stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    printf("%d\n", sizeof(arr));
    return 0;
}
```

此时输出结果是40，表示整个数组此时的占内存空间的大小为40字节。

我们可以知道个数组的字节大小为40字节，每个元素的大小为4个字节，此时就可以计算出数组的长度。

```
#include<stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    printf("%d\n", sizeof(arr)/sizeof(arr[0]));
    return 0;
}
```

此时输出结果为10，表示整个数组的长度为10。

6. 一维数组习题练习

1. 数组倒序

把数组当中的元素进行逆置输出

```
#include<stdio.h>
int main()
{
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int len = sizeof(arr) / sizeof(arr[0]);
    int i = 0;
    int j = len - 1;

    while (i < j)
    {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        i++;
        j--;
    }

    for (int i = 0; i < len; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

2. 数组元素扩大10倍

给定指定数组，将数组当中的每个元素扩大10倍，输出最后数组的元素。

例如数组数据arr：1 2 3 4扩大10倍后数组为：10 20 30 40

```
#include<stdio.h>
int main()
{
    int arr[4] = {1,2,3,4};
    int len = sizeof(arr) / sizeof(arr[0]);

    for (int i = 0; i < len; i++)
    {
        arr[i] = arr[i] * 2;
    }

    for (int i = 0; i < len; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

3. 二分查找

题目：给定一个升序的整型数组，在这个数组中查找指定的数字，找到了就打印一的下标，找不到就打印一找不到。

提示：在一个升序的数组中查找指定的数字。你应该思想的方式是是最简单的，但是这种方法效率比较低。

```
#include<stdio.h>
int main()
{
    int arr[] = {1,2,3,4,5,6,7,8,9,10};
    int left = 0;
    int right = sizeof(arr)/sizeof(arr[0])-1;
    int key = 7;//要找的数字
    int mid = 0;//记录中间元素的下标
    int find = 0;
    while(left <= right)
    {
        mid = (left+right)/2;
        if(arr[mid] > key)
        {
            right = mid-1;
        }
        else if(arr[mid] < key)
        {
            left = mid+1;
        }
        else
        {
            find = 1;
            break;
        }
    }

    if(1 == find)
        printf("找到了，下标是%d\n", mid);
    else
        printf("找不到\n");
    return 0;
}
```

优化提示： 多中间元素的下标，使用 mid = left + right / 2，如果left和right值非常大时，它们的和可能会超出整型范围导致溢出，更稳定的写法是：**mid = left + (right - left) / 2**。

📋 本节总结

- 一维数组的概念和特性
- 数组的创建及初始化方法
- 数组下标的规则及使用规则
- 数组元素的访问、输入和输出
- 数组在内存中的连续存储特性
- sizeof计算数组长度的原理和技巧
- 数组的实际应用：逆置、元素处理和二分查找